

LAB MANUAL



**Faculty of Computing, Engineering
and The Built Environment**

COM682 - Lab 07

- 1. Create Integration Workflow with Multi-tenant Azure Logic Apps**
- 2. Create a Function in Azure Portal**
- 3. Create a Personalized Function in Azure Portal**

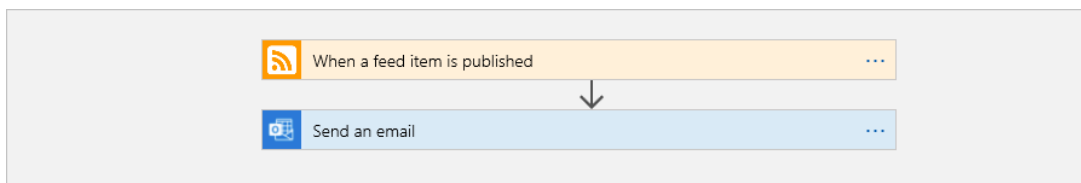
Create Integration Workflow with Multi-tenant Azure Logic Apps

Preamble

This practical shows how to create an **automated workflow that integrates two services, an RSS feed for a website and an email account**, when using multi-tenant Azure Logic Apps. While this practical is cloud-based, Azure Logic Apps supports workflows that connect apps, data, services, and systems across cloud, on-premises, and hybrid environments.

Here you create a logic app resource and workflow that uses the RSS connector and the Office 365 Outlook connector. The resource runs in multi-tenant Azure Logic Apps and is based on the Consumption pricing model. **The RSS connector has a trigger that checks an RSS feed based on a schedule. The Office 365 Outlook connector has an action that sends an email for each new item.** The connectors in this practical are only two among the hundreds of connectors that you can use in a workflow.

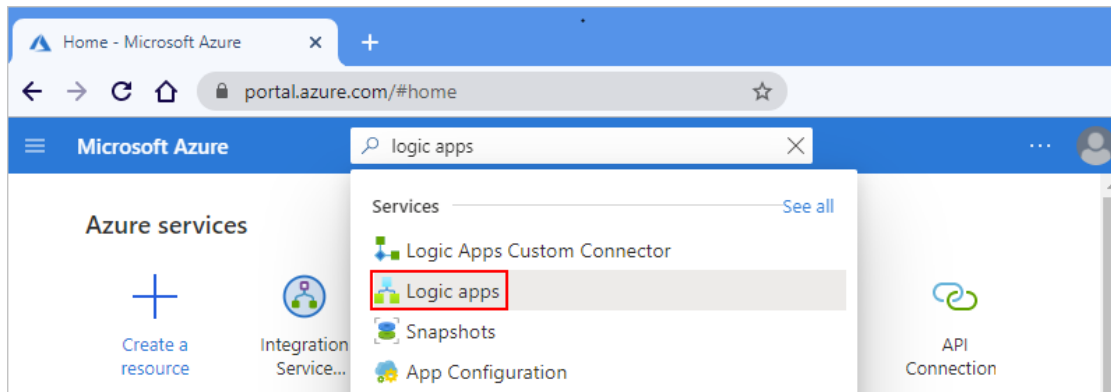
The following screenshot shows the high-level workflow:



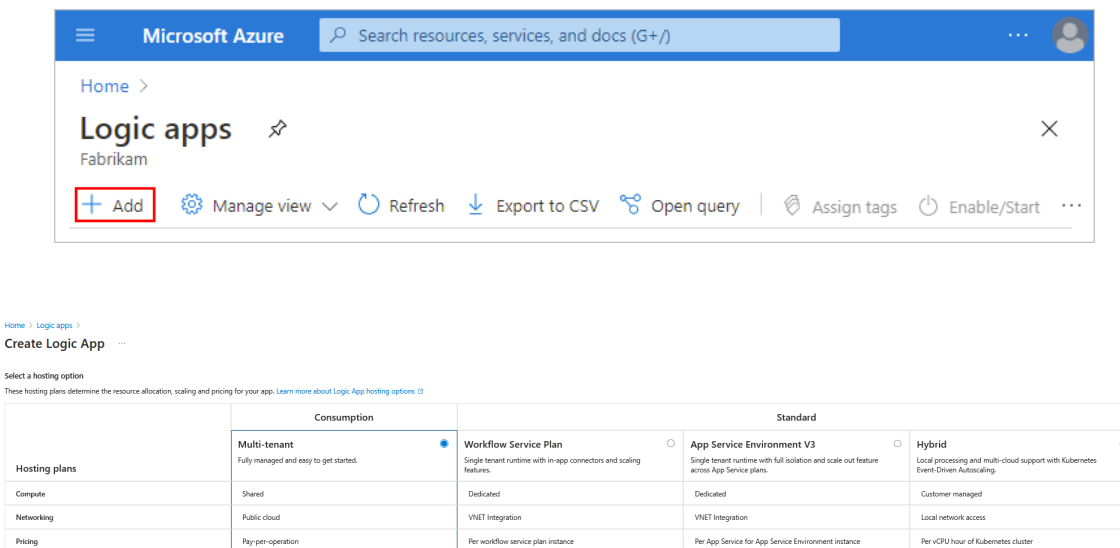
As you progress, you'll learn these basic steps:

- Create a logic app resource that runs in the multi-tenant Azure Logic Apps environment.
- Select the blank logic app template.
- Add a trigger that specifies when to run the workflow.
- Add an action that performs a task after the trigger fires.
- Run your workflow.

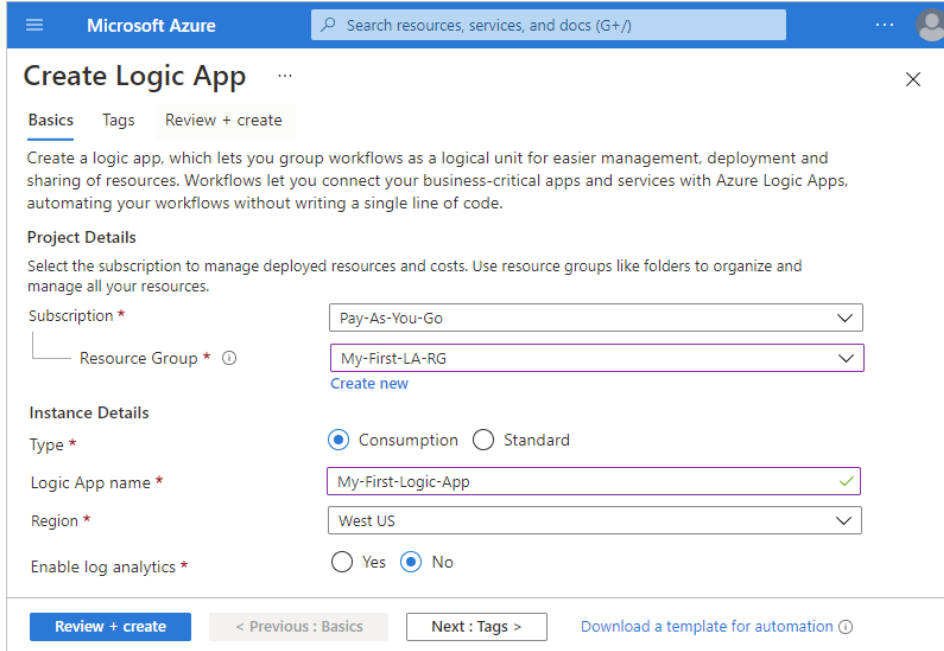
1. In the Azure search box, enter logic apps, and select **Logic apps**.



2. On the **Logic apps** page, select **Add**.



3. On the **Create Logic App** pane, select the Azure subscription to use 'Azure for Students', create a new resource group for your logic app resource, and provide basic details about your logic app resource.



Microsoft Azure Search resources, services, and docs (G+)

Create Logic App

Basics Tags Review + create

Create a logic app, which lets you group workflows as a logical unit for easier management, deployment and sharing of resources. Workflows let you connect your business-critical apps and services with Azure Logic Apps, automating your workflows without writing a single line of code.

Project Details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Pay-As-You-Go
Resource Group * My-First-LA-RG [Create new](#)

Instance Details

Type * ☒ Consumption ☐ Standard

Logic App name * My-First-Logic-App ✓

Region * West US

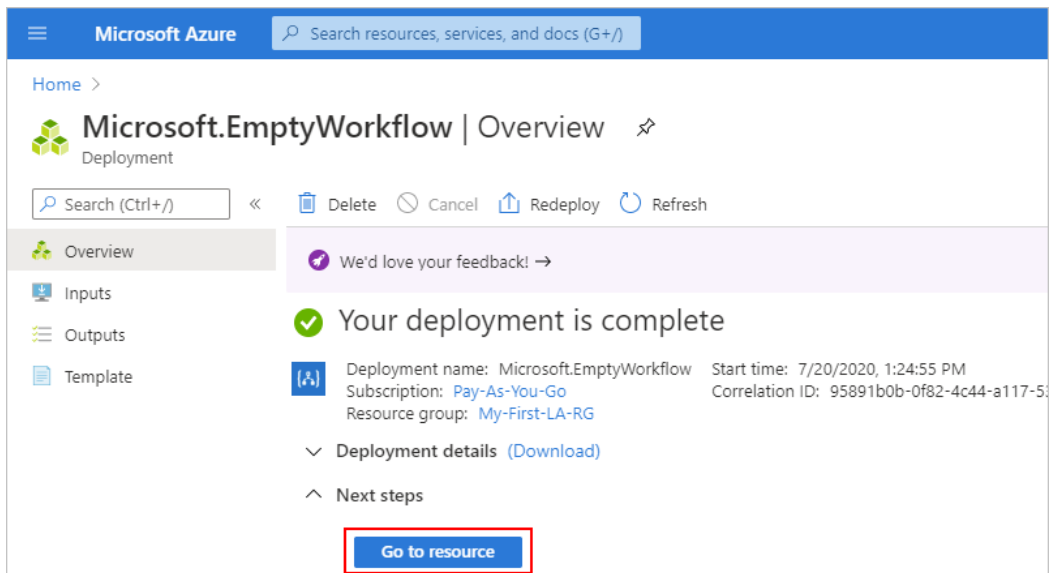
Enable log analytics * ☐ Yes ☒ No

[Review + create](#) < Previous : Basics Next : Tags > [Download a template for automation](#)

- When you're ready, select **Review + Create**. On the validation page, confirm the details that you provided, and select **Create**.

Select the Blank Template

- After Azure successfully deploys your app, select **Go to resource**. Or find and select your logic app resource by typing the name in the Azure search box.



Microsoft Azure Search resources, services, and docs (G+)

Home > **Microsoft.EmptyWorkflow | Overview**

Deployment

Search (Ctrl+/) << Delete Cancel Redeploy Refresh

Overview

Inputs

Outputs

Template

We'd love your feedback! →

✓ **Your deployment is complete**

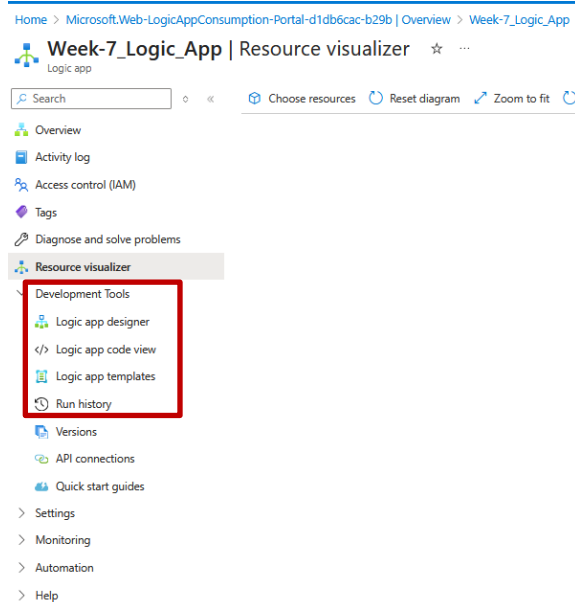
Deployment name: Microsoft.EmptyWorkflow Start time: 7/20/2020, 1:24:55 PM
Subscription: Pay-As-You-Go Correlation ID: 95891b0b-0f82-4c44-a117-50
Resource group: My-First-LA-RG

Deployment details (Download)

Next steps

[Go to resource](#)

- Under Development tools, select **Logic App Templates**, select **Blank workflow**.



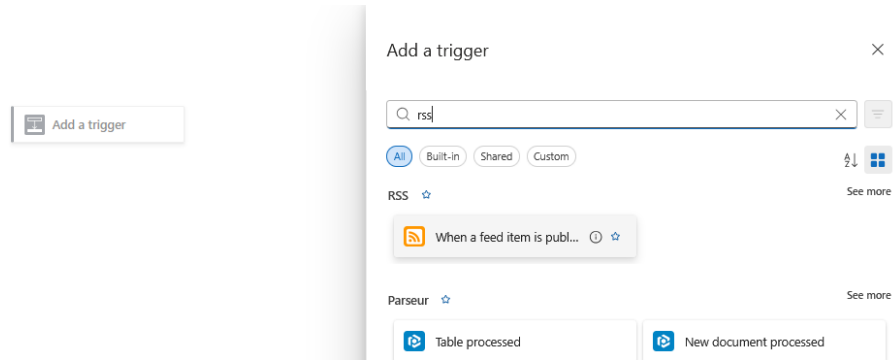
After you select the template, the designer now shows an empty workflow surface.

Add the Trigger

A workflow always starts with a single trigger, which specifies the condition to meet before running any actions in the workflow. Each time the trigger fires, Azure Logic Apps creates and runs a workflow instance. If the trigger doesn't fire, no instance is created nor run. You can start a workflow by choosing from many different triggers.

This practical uses an RSS trigger that checks an RSS feed based on a schedule. If a new item exists in the feed, the trigger fires, and a new workflow instance runs. If multiple new items exist between checks, the trigger fires for each item and a separate new workflow instance runs for each item.

1. Click on 'Add a trigger'
2. To find the RSS trigger, in the search box, enter **rss**. From the **Triggers** list, select the RSS trigger, **When a feed item is published**.



3. In the trigger details, provide the following information
<https://feeds.a.dj.com/rss/RSSMarketsMain.xml>


When a feed item is published
⋮
×

▼ Add a description

Parameters
Settings
Code view
About

The RSS feed URL *

Advanced parameters

Showing 1 of 1
 Show all
Clear all

Chosen property will be used to determine which items are new.

PublishDate
 ×

How often do you want to check for items?

Recurrence *

Interval *

Frequency *

Minute

Time zone

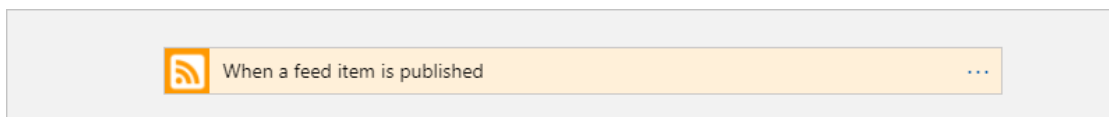
(UTC+00:00) Dublin, Edinburgh, Lisbon, London

Start time

Example: 2017-03-24T15:00:00Z

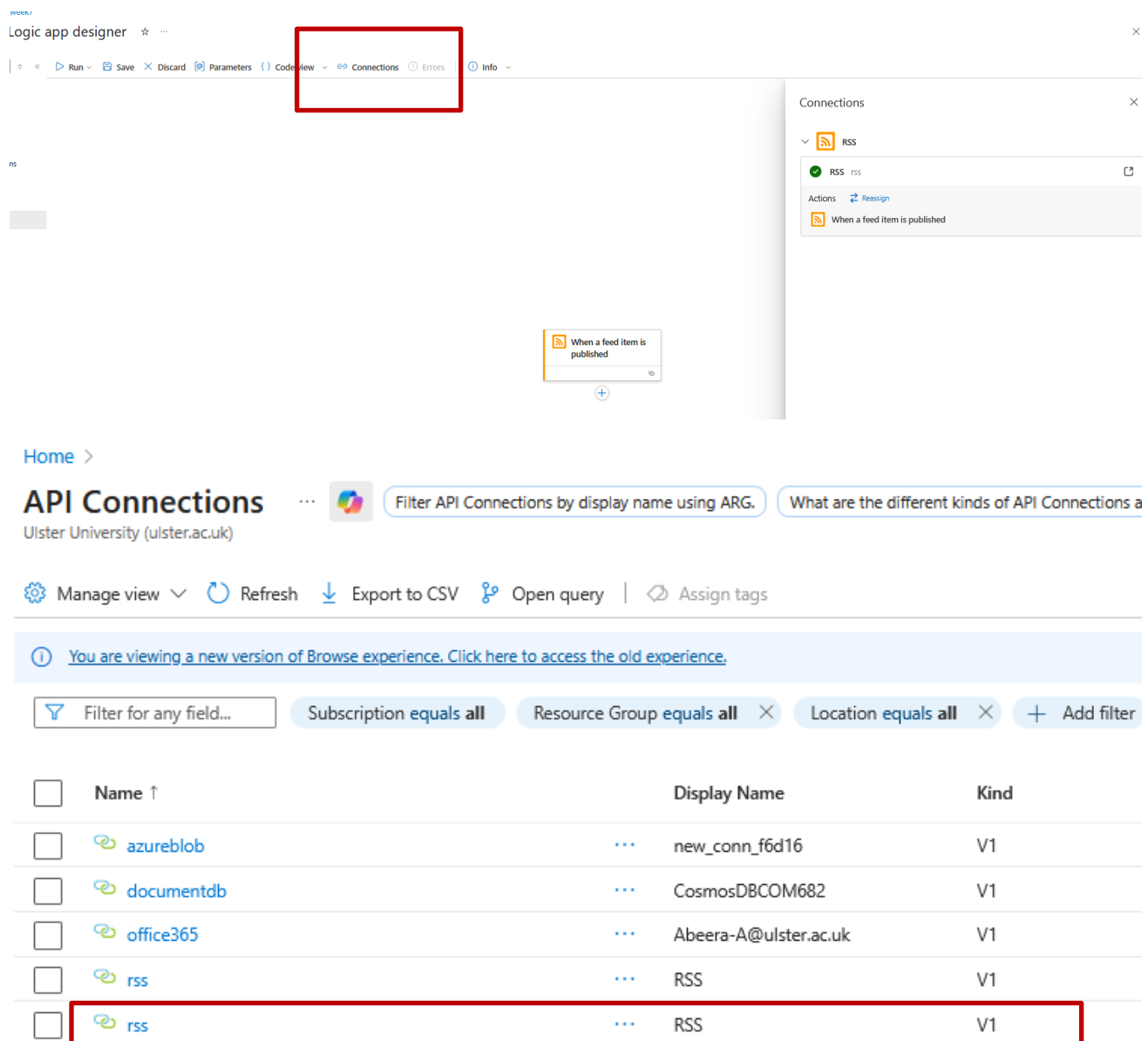
Connected to rss.
Change connection

4. Collapse the trigger's details for now by clicking inside its title bar.



- When you're done, save your logic app, which instantly goes live in the Azure portal. On the designer toolbar, select **Save**.

If you receive an error, (i) click on connections and see if RSS connection is successful. Click on 'Reassign' button. (ii) Also search for API connections in another browser window, and wait till you see 'rss' named API connection available. It may take few minutes, depending upon your eligible region.



The screenshot shows the Logic App Designer interface. The 'Connections' pane on the right lists the 'RSS' connection. The main canvas shows a trigger 'When a feed item is published'. Below this, the 'API Connections' page is displayed, showing a list of connections. The 'rss' connection is highlighted with a red box.

Name	Display Name	Kind
azureblob	new_conn_f6d16	V1
documentdb	CosmosDBCOM682	V1
office365	Abeera-A@ulster.ac.uk	V1
rss	RSS	V1
rss	RSS	V1

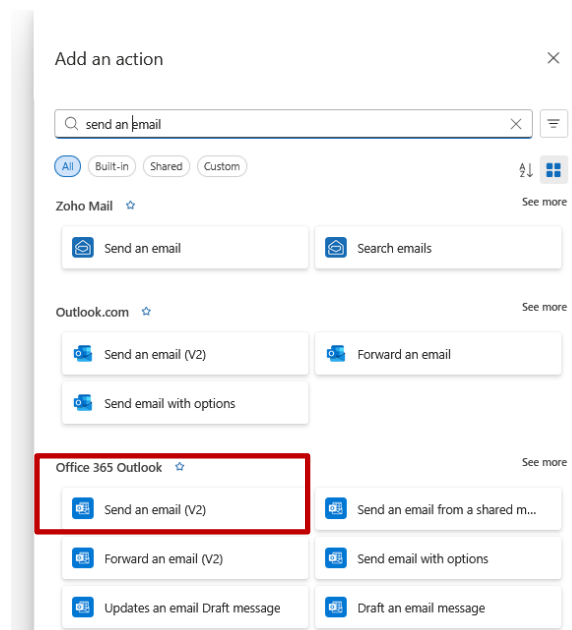
The trigger won't do anything other than check the RSS feed. So, you need to add an action that defines what happens when the trigger fires.

Add an Action

Following a trigger, an action is a subsequent step that runs some operation in the workflow. Any action can use the outputs from the previous step, which can be the trigger or another action. You can choose from many different actions, add multiple actions up to the limit per workflow, and even create different action paths.

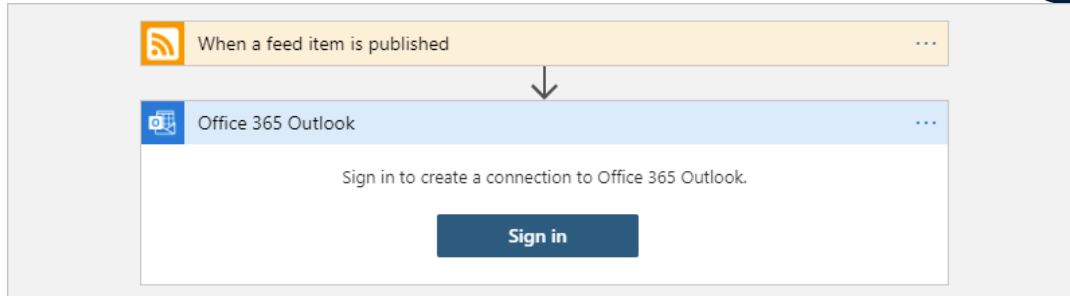
This example uses an Office 365 Outlook action that sends an email each time the trigger fires for a new RSS feed item. If multiple new items exist between checks, you receive multiple emails.

1. Under the **When a feed item is published** trigger, select **+**.
2. In the search box, enter **send an email** so that you can find connectors that offer this action. We used **Office 365 Send an email (v2)**

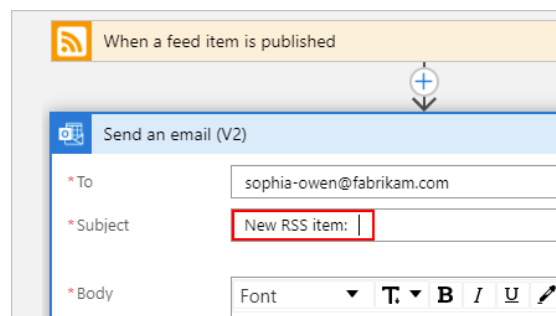


3. If your selected email service prompts you to sign in and authenticate your identity, complete that step now.

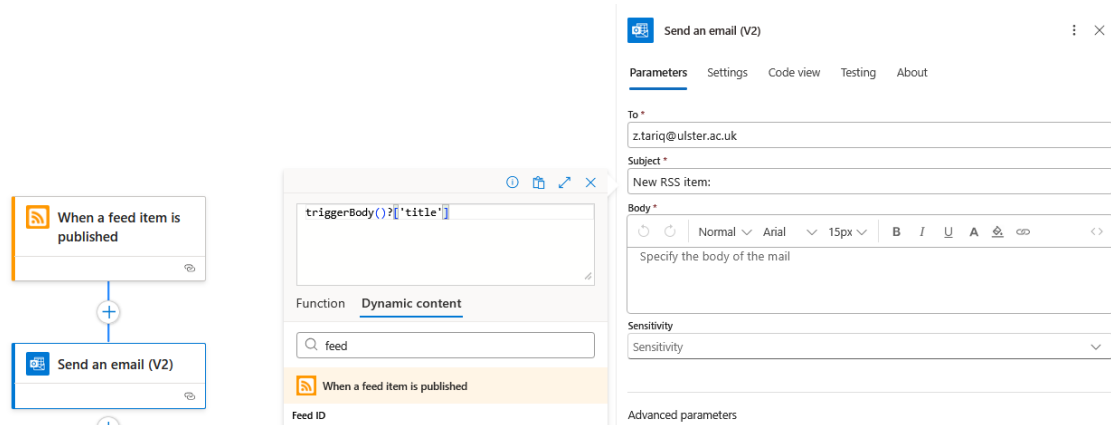
Many connectors require that you first create a connection and authenticate your identity before you can continue.



4. In the **Send an email** action, specify the information to include in the email.
 - In the **To** box, enter the receiver's email address. For this example, use your email address.
 - In the **Subject** box, enter the email subject. For this example, enter the following text with a trailing blank space: **New RSS item:**

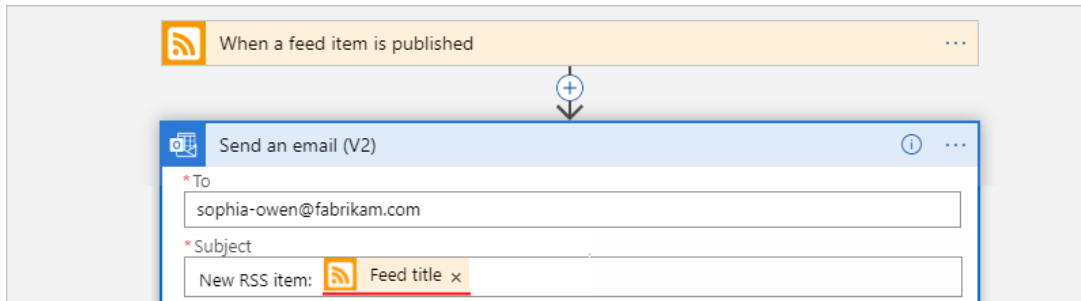


- From the **Add dynamic content** list, under **When a feed item is published**, select **Feed title**.



- The feed title is a trigger output that references the title for the RSS item. Your email uses this output to show the RSS item's title.

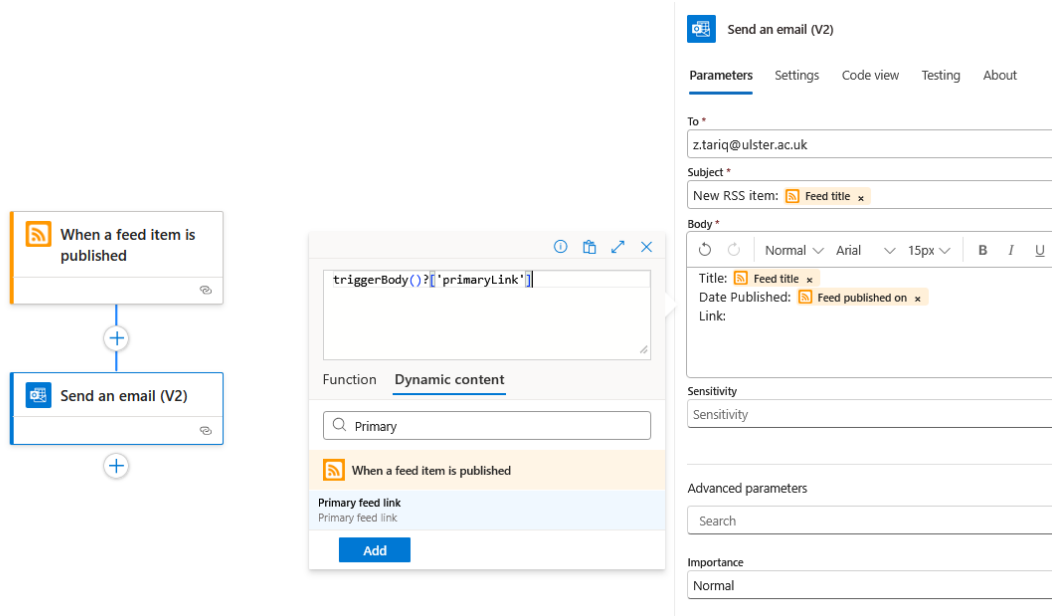
When you're done, the email subject looks like this example:



- In the Body box, enter email body content.

For this example, the body includes the following properties, preceded by descriptive text for each property. To add blank lines in an edit box, press **Shift + Enter**.

Descriptive text	Property	Description
Title:	Feed title	The item's title
Date published:	Feed published on	The item's publishing date and time
Link:	Primary feed link	The URL for the item



The screenshot shows a workflow with two steps: 'When a feed item is published' and 'Send an email (V2)'. The 'Send an email (V2)' step is selected, and its configuration panel is open. The panel shows the following settings:

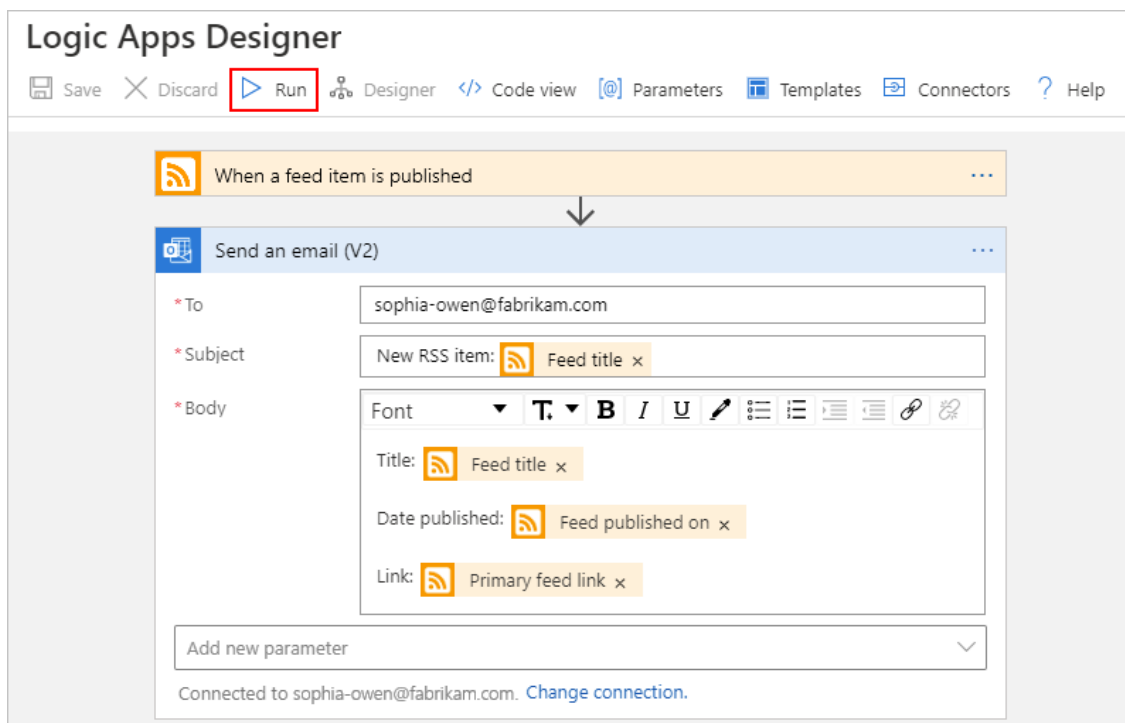
- To:** z.tariq@ulster.ac.uk
- Subject:** New RSS item: Feed title
- Body:**
 - Title: Feed title
 - Date Published: Feed published on
 - Link:
- Sensitivity:** Sensitivity
- Advanced parameters:** Search
- Importance:** Normal

The 'Dynamic content' panel is also open, showing the 'Primary feed link' property being added to the body content.

5. Save your logic app. On the designer toolbar, select **Save**.

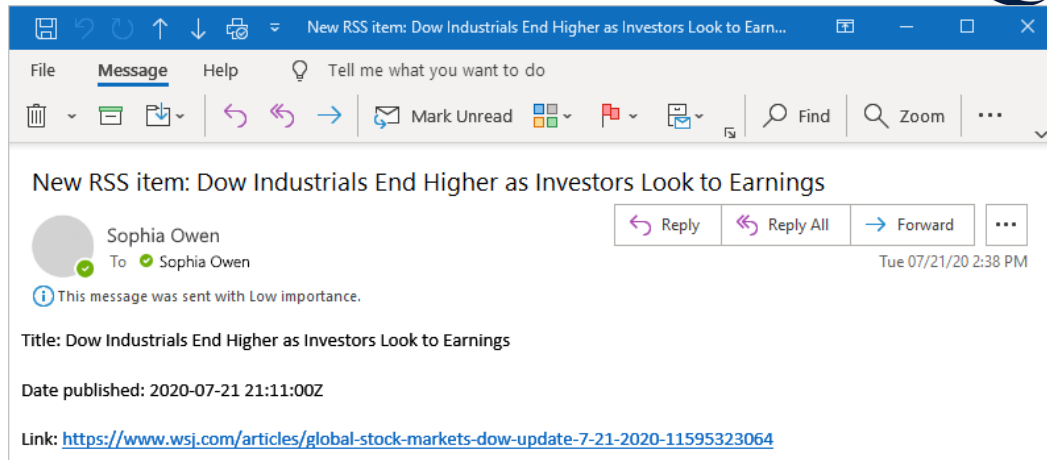
Run your Workflow

To check that the workflow runs correctly, you can wait for the trigger to check the RSS feed based on the set schedule. Or, you can manually run the workflow by selecting **Run** on the workflow designer toolbar, as shown in the following screenshot.



If the RSS feed has new items, your workflow sends an email for each new item. Otherwise, your workflow waits until the next interval to recheck the RSS feed.

The following screenshot shows a sample email that's sent by the example workflow. The email includes the details from each trigger output you selected, plus the descriptive text you included for each item.



Troubleshoot Problems

If you don't receive emails from the workflow as expected:

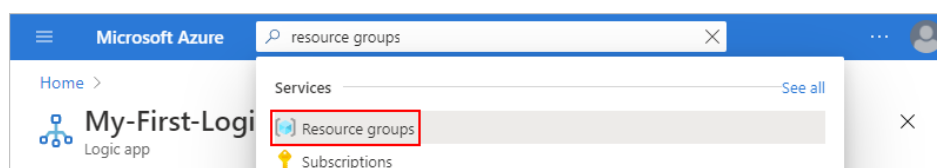
- Check your email account's junk or spam folder in case the message was incorrectly filtered.
- Make sure the RSS feed you're using has published items since the last scheduled or manual check.

Note that waiting for an RSS to be published might take some time. So, once you have completed the practical, feel free to let the Logic App running and check your email inbox tomorrow morning. Once you have confirmed this practical works, proceed to clean up your Azure Resources as instructed below.

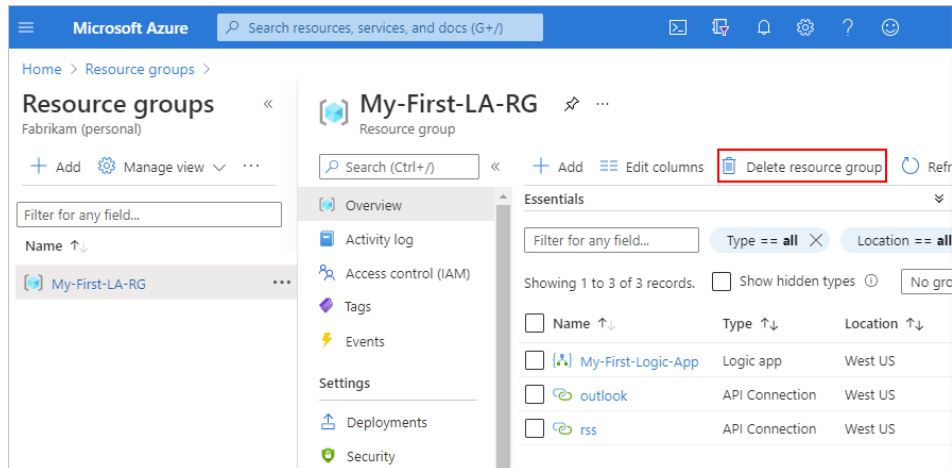
Clean up Resources

When you're done with this practical, delete the sample logic app resource and any related resources by deleting your created resource group.

1. In the Azure search box, enter resource groups and then select **Resource groups**.



2. Find and select your logic app's resource group. On the **Overview** pane, select **Delete resource group**.



3. When the confirmation page appears, enter the resource group name, and select **Delete**.

Create a Function in Azure Portal

Preamble

In this practical, you use Visual Studio Code to create a Python function that responds to HTTP requests. After testing the code locally, you deploy it to the serverless environment of Azure Functions.

Configure your environment

Before you get started, make sure you have the following requirements in place:

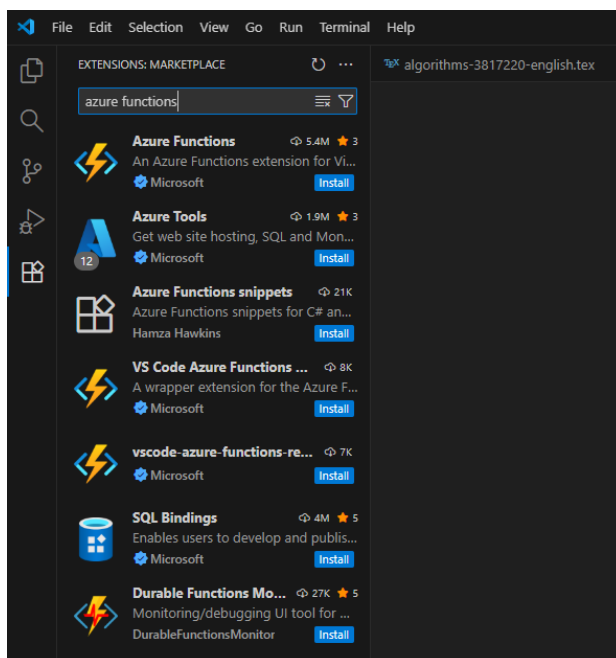
- The [Azure Functions Core Tools](#) version 4.x.
- The [Python extension](#) for Visual Studio Code.
- The [Azure Functions extension](#) for Visual Studio Code.

Create your local project

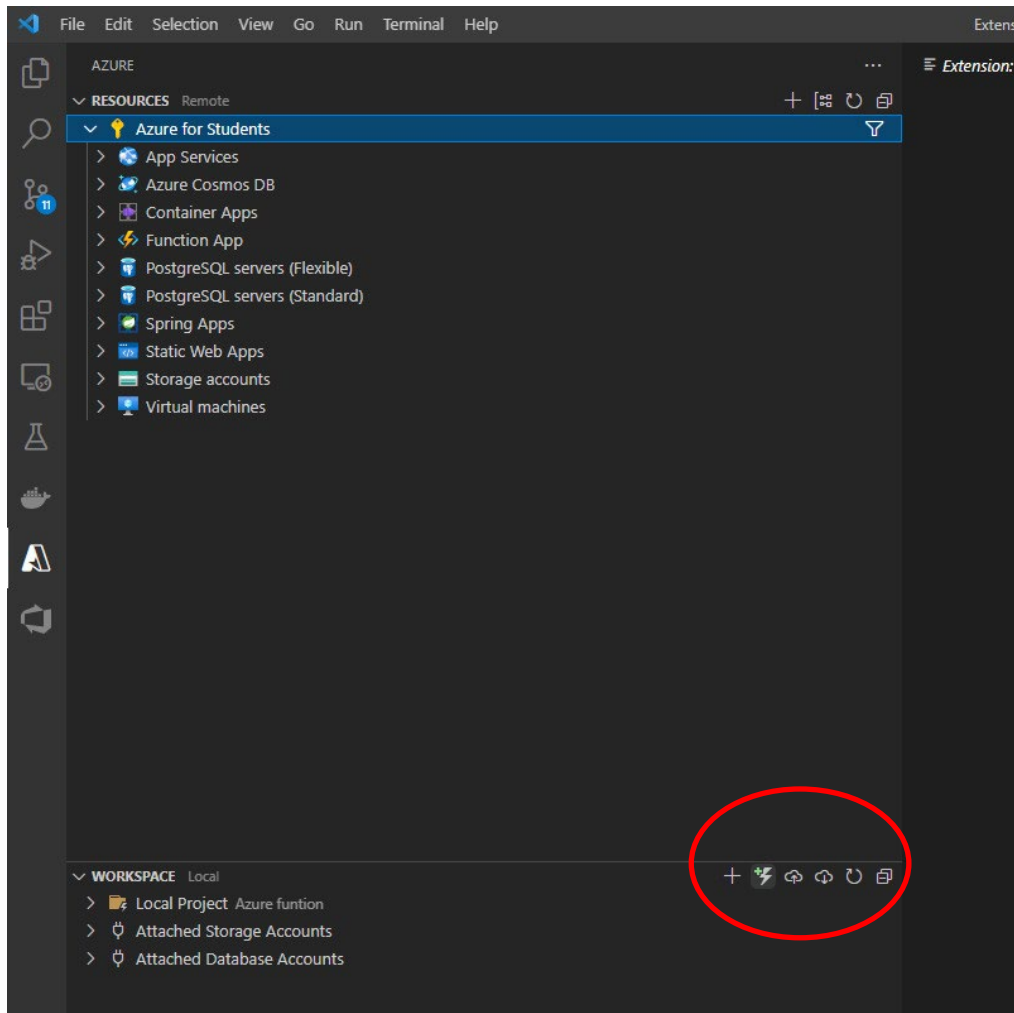
In this section, you use Visual Studio Code to create a local Azure Functions project in Python. Later in this practical, you'll publish your function code to Azure.

1. In Visual Studio Code, press F1 to open the command palette and search for and run the command Azure Functions: Create New Project.... Select the directory location for your project workspace, and then choose **Select**.

If Azure functions are not installed, you can install through Marketplace as below:



Choose the Azure icon in the Activity bar, then in the **Azure: Functions** area, select the **Create new project...** icon.



Choose a directory location for your project workspace and choose **Select**.

Provide the following information at the prompts:

- **Select a language for your function project:** Choose **Python**.
- **Select a Python alias to create a virtual environment:** Choose the location of your Python interpreter.
If the location isn't shown, type in the full path to your Python binary.
- **Select a template for your project's first function:** Choose **HTTP trigger**.
- **Provide a function name:** Type **HttpExample**.
- **Authorization level:** Choose **Anonymous**, which enables anyone to call your function endpoint.
- **Select how you would like to open your project:** Choose **Add to workspace**.

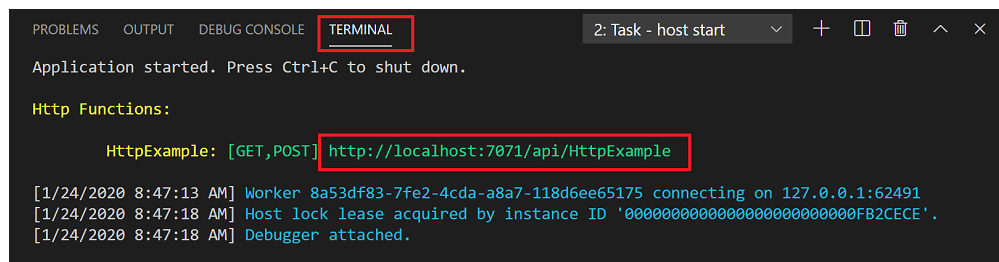
Functions lets you use keys to make it harder to access your HTTP function endpoints during development. Unless the HTTP access level on an HTTP triggered function is set to anonymous, requests must include an API access key in the request.

Using this information, Visual Studio Code generates an Azure Functions project with an HTTP trigger. You can view the local project files in the Explorer.

Run the function locally

Visual Studio Code integrates with Azure Functions Core tools to let you run this project on your local development computer before you publish to Azure.

1. To call your function, press **F5** to start the function app project. Output from Core Tools is displayed in the **Terminal** panel. Your app starts in the **Terminal** panel. You can see the URL endpoint of your HTTP-triggered function running locally.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 2: Task - host start
Application started. Press Ctrl+C to shut down.

Http Functions:

    HttpExample: [GET,POST] http://localhost:7071/api/HttpExample

[1/24/2020 8:47:13 AM] Worker 8a53df83-7fe2-4cda-a8a7-118d6ee65175 connecting on 127.0.0.1:62491
[1/24/2020 8:47:18 AM] Host lock lease acquired by instance ID '00000000000000000000000000000000FB2CECE'.
[1/24/2020 8:47:18 AM] Debugger attached.
```

If you have trouble running on Windows, make sure that the default terminal for Visual Studio Code isn't set to **WSL Bash**.

2. With Core Tools running, go to the **Azure: Functions** area. Under **Functions**, expand **Local Project > Functions**. Right-click (Windows) or **Ctrl** - click (macOS) the **HttpExample** function and choose **Execute Function Now....**

- ```
✓ Show welcome page on startup

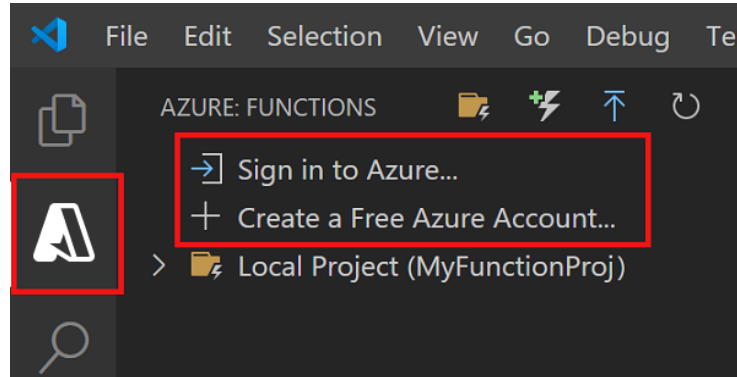
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

func: host start - Task ✓ + ▢ ▢ ... | 🔍

For detailed output, run func with --verbose flag.

[2025-11-01T12:04:37.613Z] Host lock lease acquired by instance ID '00000000000000000000000011194A6F'.
[2025-11-01T13:17:24.230Z] Executing 'Functions.HttpExample' (Reason='This function was programmatically called via the host APIs.', Id=2a2534c7-65d8-4a7b-a134-cfc0f6ae32f2)
[2025-11-01T13:17:24.293Z] Python HTTP trigger function processed a request.
[2025-11-01T13:17:24.316Z] Executed 'Functions.HttpExample' (Succeeded, Id=2a2534c7-65d8-4a7b-a134-cfc0f6ae32f2, Duration=108ms)
[2025-11-01T13:17:37.943Z] Executing 'Functions.HttpExample' (Reason='This function was programmatically called via the host APIs.', Id=6d9cf6f4-cd2c-4a8e-96bc-ebba0361eb4a)
[2025-11-01T13:17:37.954Z] Python HTTP trigger function processed a request.
[2025-11-01T13:17:37.957Z] Executed 'Functions.HttpExample' (Succeeded, Id=6d9cf6f4-cd2c-4a8e-96bc-ebba0361eb4a, Duration=20ms)
[2025-11-01T13:20:09.631Z] Executing 'Functions.HttpExample' (Reason='This function was programmatically called via the host APIs.', Id=50863a69-00dc-4b07-9421-e1caf5cf646b)
[2025-11-01T13:20:09.637Z] Python HTTP trigger function processed a request.
[2025-11-01T13:20:09.638Z] Executed 'Functions.HttpExample' (Succeeded, Id=50863a69-00dc-4b07-9421-e1caf5cf646b, Duration=10ms)
```

1. If you aren't already signed in, choose the Azure icon in the Activity bar, then in the **Azure: Functions** area, choose **Sign in to Azure**....



If you're already signed in, go to the next section.

2. When prompted in the browser, choose your Azure account and sign in using your Azure account credentials.
3. After you've successfully signed in, you can close the new browser window. The subscriptions that belong to your Azure account are displayed in the Side bar.

### Publish the project to Azure

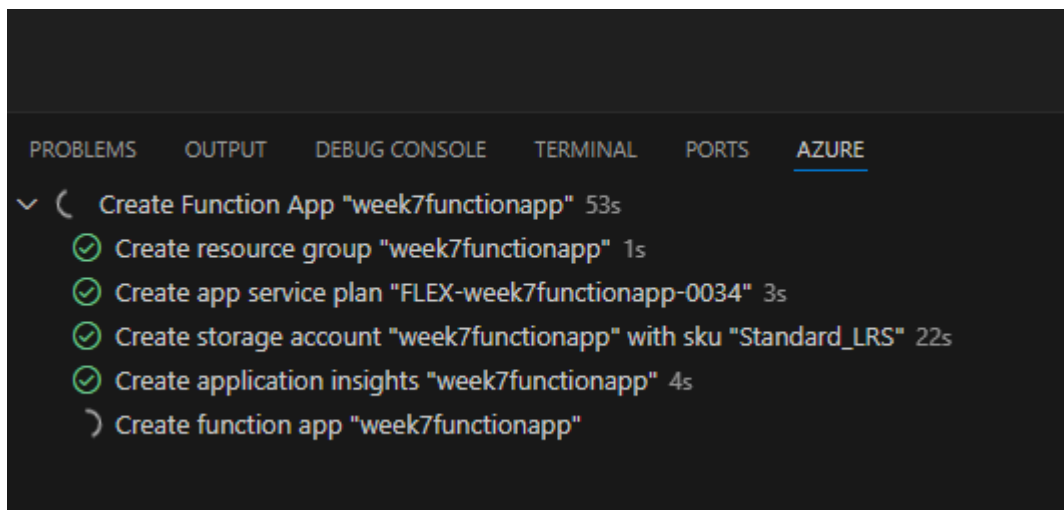
In this section, you create a function app and related resources in your Azure subscription and then deploy your code.

1. Choose the Azure icon in the Activity bar, then in the **Azure: Functions** area, right click on Local Project and choose the **Deploy to function app...** button.
2. Provide the following information at the prompts:
  - **Select subscription:** Choose the subscription to use.  
*You won't see this if you only have one subscription.*
  - **Select Function App in Azure:** Choose **+ Create new Function App**.  
*(Don't choose the **Advanced** option, which isn't covered in this practical.)*
  - **Enter a globally unique name for the function app:** Type a name that is valid in a URL path. *The name you type is validated to make sure that it's unique in Azure*

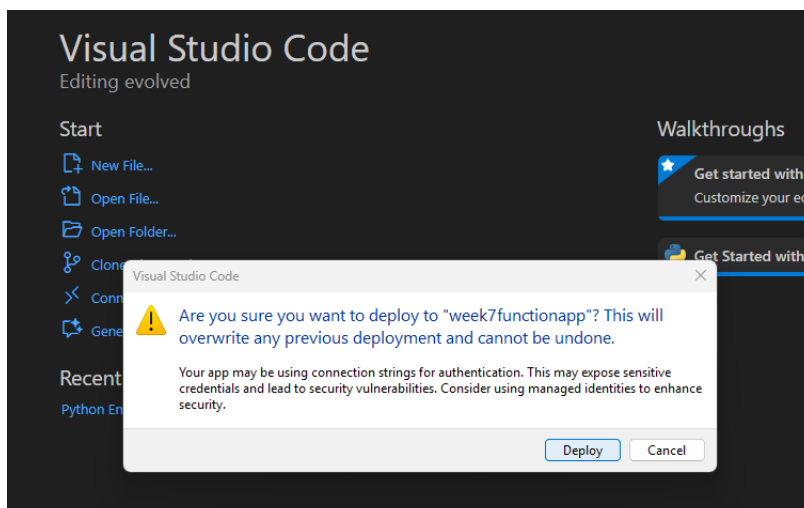
### *Functions.*

- **Select a location for new resources:** For better performance, choose one which is available for you .
- **Select a runtime:** Choose the version of Python you've been running on locally. *You can use the `python --version` command to check your version.*
- Next option selects *Secrets*

The extension shows the status of individual resources as they are being created in Azure in the notification area.



If given following prompt, select Deploy

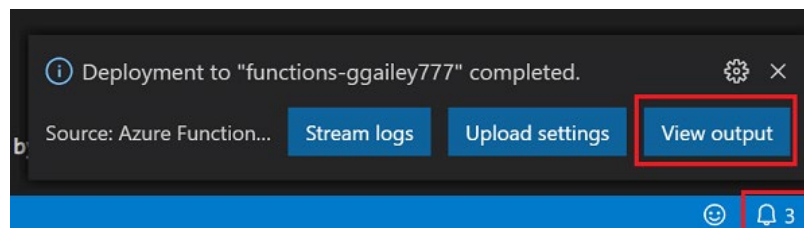


3. When completed, the following Azure resources are created in your subscription, using names based on your function app name:

- A resource group, which is a logical container for related resources.
- A standard Azure Storage account, which maintains state and other information about your projects.
- A consumption plan, which defines the underlying host for your serverless function app.
- A function app, which provides the environment for executing your function code. A function app lets you group functions as a logical unit for easier management, deployment, and sharing of resources within the same hosting plan.
- An Application Insights instance connected to the function app, which tracks usage of your serverless function.

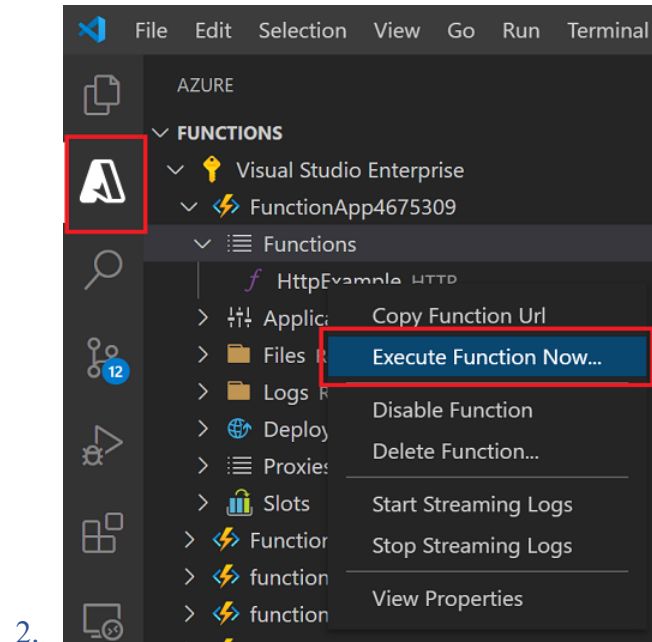
A notification is displayed after your function app is created and the deployment package is applied.

4. Select **View Output** in this notification to view the creation and deployment results, including the Azure resources that you created. If you miss the notification, select the bell icon in the lower right corner to see it again.



### Run the function in Azure

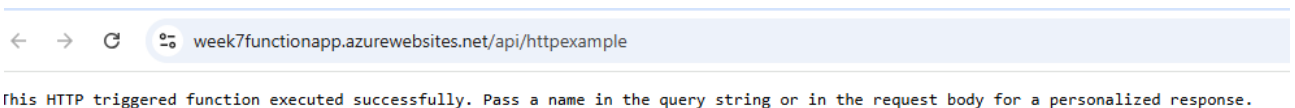
1. Back in the **Azure: Functions** area in the side bar, expand your subscription, your new function app, and **Functions**. Right-click (Windows) or **Ctrl** - click (macOS) the **HttpExample** function and choose **Execute Function Now....**



3. In **Enter request body** you see the request message body value of `{ "name": "Jhon" }`. Press Enter to send this request message to your function.

4. When the function executes in Azure and returns a response, a notification is raised in Visual Studio Code.

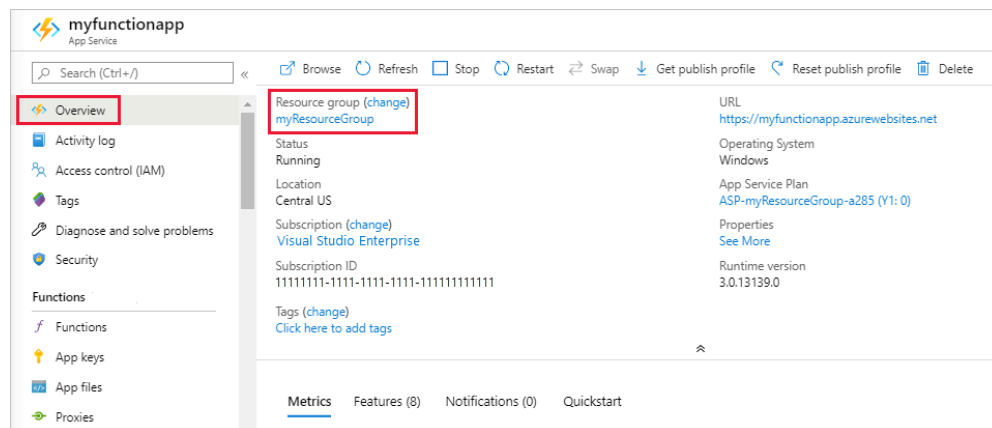
If you click on the link, you can see your function deployed on Azure like below:



## Clean up resources

You can use the following steps to delete the function app and its related resources to avoid incurring any further costs:

1. In Visual Studio Code, press **F1** to open the command palette. In the command palette, search for and select **Azure Functions: Open in portal**.
2. Choose your function app, and press **Enter**. The function app page opens in the Azure portal.
3. In the **Overview** tab, select the named link next to **Resource group**.



4. In the **Resource group** page, review the list of included resources, and verify that they are the ones you want to delete.
5. Select **Delete resource group** and follow the instructions.
6. Deletion may take a couple of minutes. When it's done, a notification appears for a few seconds. You can also select the bell icon at the top of the page to view the notification.

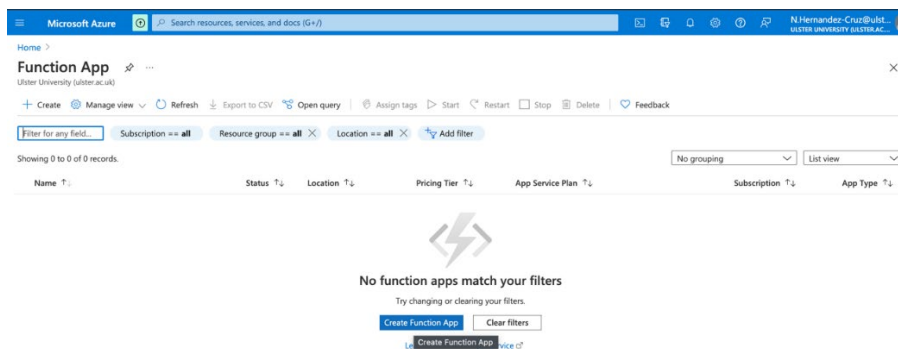
# Create a Personalized Function in Azure Portal

## Preamble

In this practical, you use Visual Studio Code to create your own Python function that responds to HTTP requests. **This practical differs from the previous one, as here you will be able to edit and personalize the function, rather than being constrained to the templates from Azure Functions.** After testing the code locally, you deploy it to the serverless environment of Azure Functions.

## Straight Forward Process

1. Open the Function App dashboard and select the button **Create Function App**.



Home > Function App > Create Function App

Select a hosting option

These options determine how your app scales, resources available per instance, and pricing. [Learn more about Functions hosting options](#)

| Hosting plans                            | Flex Consumption           | Functions Premium              | App Service                    | Container Apps environment     | Consumption  |
|------------------------------------------|----------------------------|--------------------------------|--------------------------------|--------------------------------|--------------|
| Scale to zero                            | ✓                          | -                              | -                              | ✓                              | ✓            |
| Scale behavior                           | Fast event-driven          | Event-driven                   | Metrics based                  | Event-driven with KEDA         | Event-driven |
| Virtual networking                       | ✓                          | ✓                              | ✓                              | ✓                              | -            |
| Dedicated compute and prevent cold start | Optional with Always Ready | Minimum of 1 instance required | Minimum of 1 instance required | Optional with minimum replicas | -            |
| Max scale out (instances)                | 1000                       | 100                            | 30                             | 300                            | 200          |

2. Fill out the formulary respectfully (for *Node.js*).
3. Select **Review + create**.
4. Select **Create**.

5. Keep everything else as by default

[Home](#) > [Function App](#) > [Create Function App](#) >

## Create Function App (Flex Consumption) ...

[Basics](#) [Storage](#) [Azure OpenAI](#) [Networking](#) [Monitoring](#) [Deployment](#) [Authentication](#) [Tags](#) [Review + create](#)

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

### Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Subscription *   | <div>Azure for Students</div>                                         |
| Resource Group * | <div>(New) week7-FA_group</div> <div><a href="#">Create new</a></div> |

### Instance Details

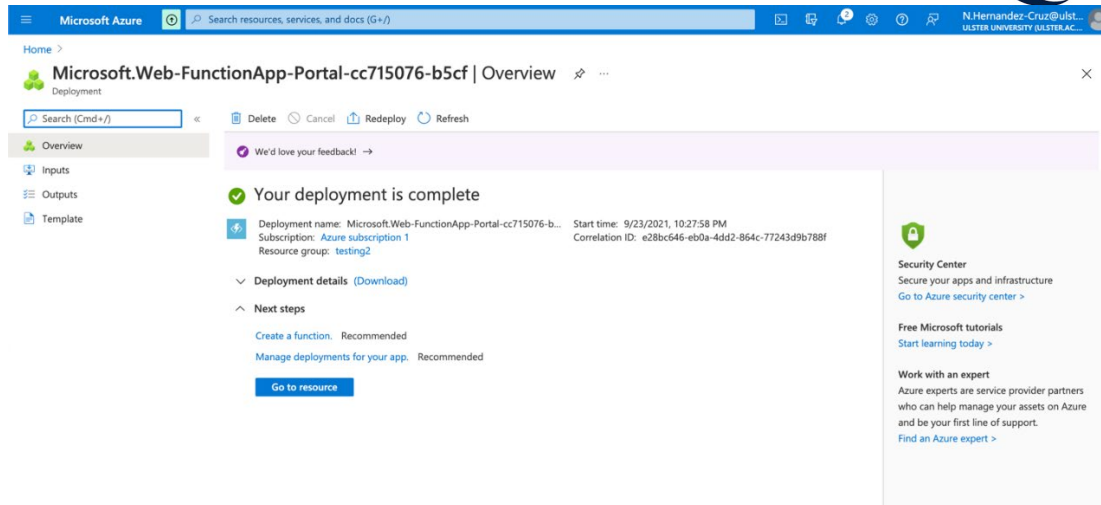
|                     |                                                                                                                                    |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Function App name * | <div>week7-FA</div> <div>.azurewebsites.net</div>                                                                                  |
|                     | <div><input checked="" type="checkbox"/> Secure unique default hostname (preview) on. <a href="#">More about this update</a></div> |
| Region *            | <div>Norway East</div>                                                                                                             |
| Runtime stack *     | <div>Node.js</div>                                                                                                                 |
| Version *           | <div>22 LTS</div>                                                                                                                  |
| Instance size *     | <div>2048 MB</div>                                                                                                                 |

### Zone redundancy

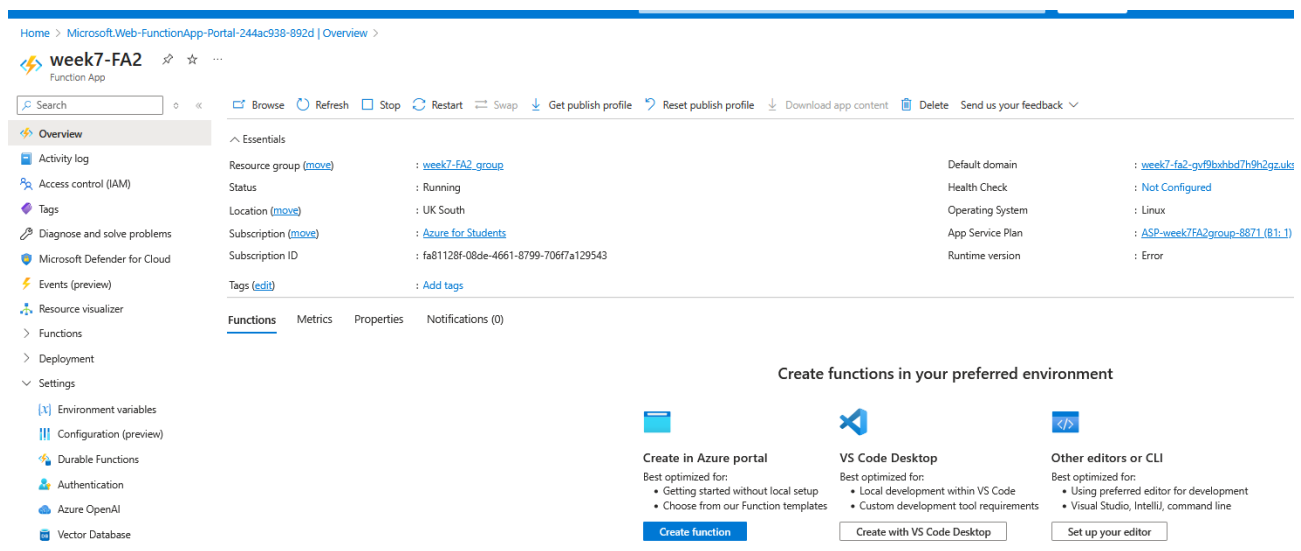
Instances of your app are distributed across availability zones for increased reliability. [More about zone redundancy.](#)

|                 |                                                                                                                                                                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zone redundancy | <div><input type="radio"/> <b>Enabled:</b> Your Flex Consumption app will be zone redundant. This changes your app's required instance count per function or function group.</div> <div><input checked="" type="radio"/> <b>Disabled:</b> Your Flex Consumption app will not be zone redundant.</div> |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

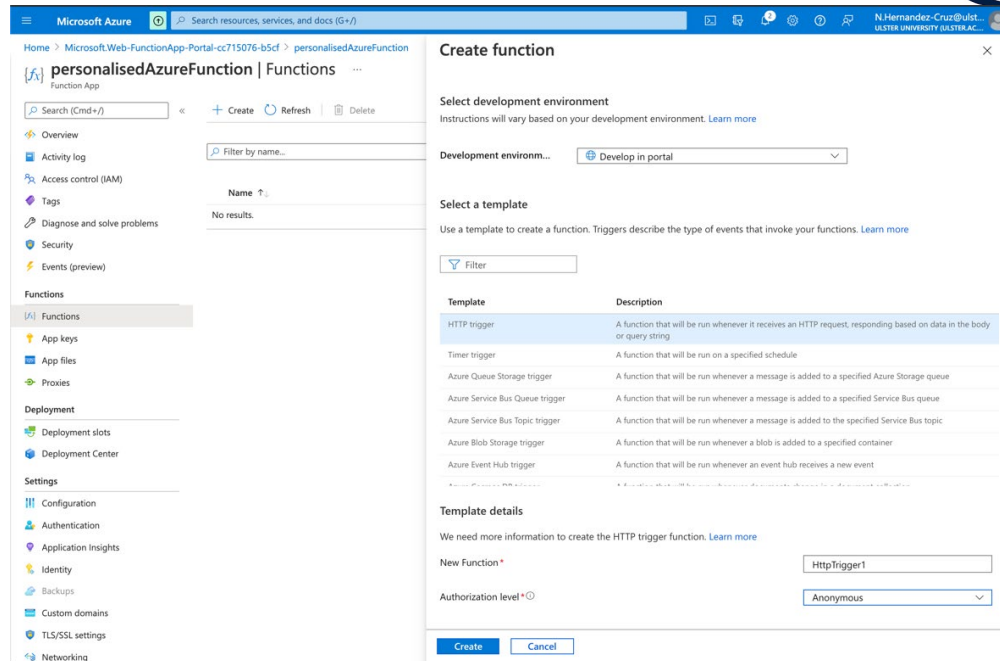
6. Select [Go to resource](#).



7. Create an HTTP Trigger function. Select the option **Functions** from the menu on the left side of the screen.

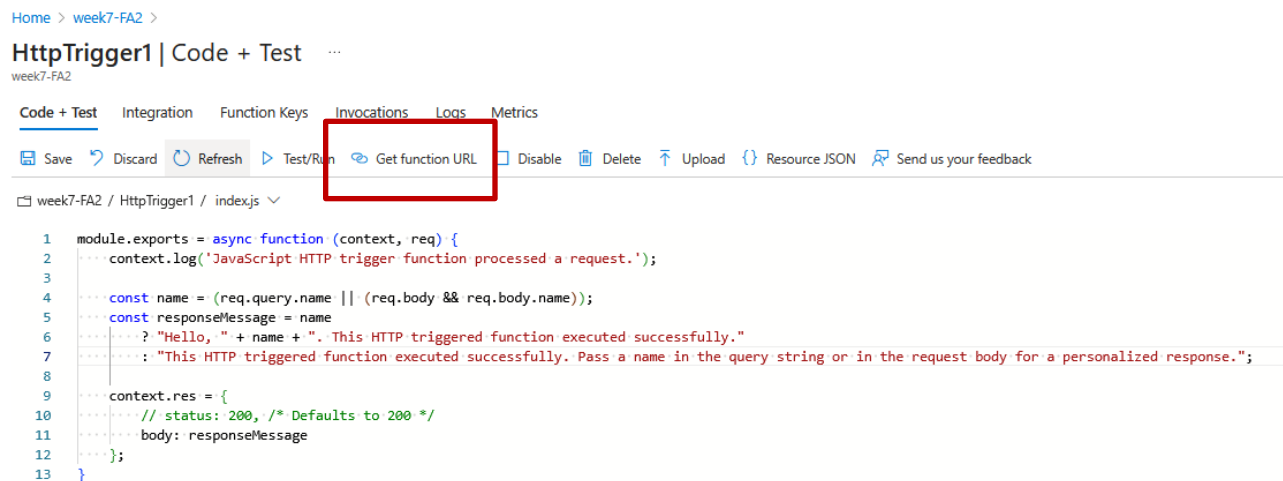


8. Select **Create Function in Azure Portal**. (If you are not seeing this option, it means its not allowed in your selected region. Create another Function app with different location, or use local function)
9. Once the formulary opens. In the field **Development environm...** select as **Develop in portal**. In template, select **HTTP trigger** (as we want a function that will run whenever it receives an HTTP request, responding based on data in the body or query string), and in **Authorization level**, select **Anonymous**.

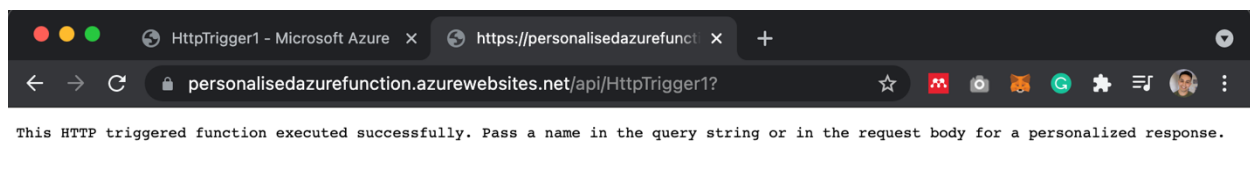


10. Select **Create**.

11. Select **Get Function Url** to see the public URL to query your function.



12. Copy the url and paste it on a net tab in your browser to query your function.



13. Finally, select the option **Logs** at the bottom of the screen and observe how every action is registered. You could use the log for debugging.

Logs

Filesystem Logs

Log Level

Stop

Copy

Clear

Maximize

Leave Feedback

Connected!

2021-09-23T21:51:19 Welcome, you are now connected to log-streaming service. The default timeout is 2 hours. Change the timeout with the App Setting SCM\_LOGSTREAM\_TIMEOUT (in seconds).
2021-09-23T21:52:15.353 [Information] Executing 'Functions.HttpTrigger1' (Reason='This function was programmatically called via the host APIs.', Id=80838e67-d6a5-41ba-87cf-eb914a4c11b9)
2021-09-23T21:52:15.418 [Information] JavaScript HTTP trigger function processed a request.
2021-09-23T21:52:15.422 [Information] Executed 'Functions.HttpTrigger1' (Succeeded, Id=80838e67-d6a5-41ba-87cf-eb914a4c11b9, Duration=70ms)
2021-09-23T21:53:19 No new trace in the past 1 min(s).
2021-09-23T21:54:19 No new trace in the past 2 min(s).
2021-09-23T21:55:19 No new trace in the past 3 min(s).
2021-09-23T21:56:19 No new trace in the past 4 min(s).
2021-09-23T21:57:19 No new trace in the past 5 min(s).
2021-09-23T21:58:19 No new trace in the past 6 min(s).
2021-09-23T21:59:19 No new trace in the past 7 min(s).

## Clean up resources

7. Select the resources you want to delete and select the **Delete**.

Microsoft Azure

Search resources, services, and docs (G+)

N.Hernandez-Cruz@ulst...

ULSTER UNIVERSITY (ULSTERAC...

Home

testing2

Resource group

Search (Cmd+)

Create

Edit columns

Delete resource group

Refresh

Export to CSV

Open query

Assign tags

Move

Delete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Deployments

Security

Policies

Properties

Locks

Cost Management

Cost analysis

Cost alerts (preview)

Budgets

Essentials

Subscription (change) : Azure subscription 1
Deployments : 2 Failed, 2 Succeeded
Subscription ID : e3040dbe-a821-45e7-bfda-224cf0b447f4
Location : UK South
Tags (change) : Click here to add tags

Resources

Recommendations

Filter for any field...
Type == all
Location == all
Add filter

Showing 1 to 6 of 6 records.
Show hidden types
No grouping
List view

| Name                      | Type                 | Location |
|---------------------------|----------------------|----------|
| ASP-testing2-8123         | App Service plan     | UK South |
| ASP-testing2-bce2         | App Service plan     | UK South |
| personalisedAzureFunction | Function App         | UK South |
| personalisedAzureFunction | Application Insights | UK South |
| personalisedFunction      | Function App         | UK South |
| personalisedFunction      | Application Insights | UK South |